Future & Promise

Future/Promise · Async thumbnail then compose URL · Build with Waqar

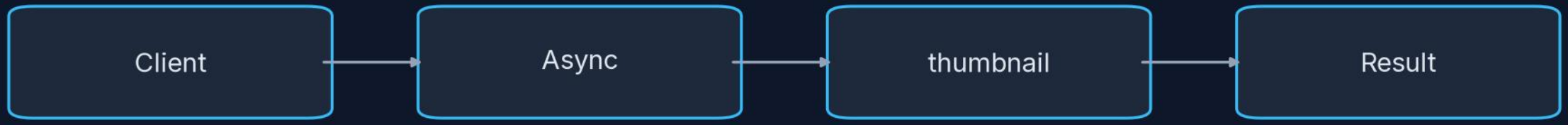
Today's Hook

- Problem: Async thumbnail then compose URL
- Pattern: Future/Promise — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Async thumbnail then compose URL
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Future/Promise: Async thumbnail then compose URL

Real-World Scenario

- Running example: Async thumbnail then compose URL
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.futurepromise.problem;

import java.util.*;

public final class NaiveThumb {
  public static String url(String id){ return "https://cdn/"+id+".jpg"; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.concurrency.futurepromise.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class ThumbService {
  CompletableFuture<String> render(String id){
    return CompletableFuture.supplyAsync(() -> {
      if (id==null || id.isBlank()) throw new IllegalArgumentException("id");
      return id+".bin";
    });
  }
  CompletableFuture<String> toUrl(String blob){ return CompletableFuture.completedFuture("https://cdn/"+blob.replace(".bin",".jpg")); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.futurepromise.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Future/Promise: async thumbnail URL ===");
    System.out.println("--- naive ---"); System.out.println(NaiveThumb.url("a"));
System.out.println("--- future ---");
var s=new ThumbService();
System.out.println(s.render("vid").thenCompose(s::toUrl).join());
try { s.render("").join(); } catch(CompletionException e){ System.out.println("caught="+e.getCause().getMessage()); }
  }
}
```

Walkthrough Output

```
=== Future/Promise: async thumbnail URL ===
--- naive ---
https://cdn/a.jpg
--- future ---
https://cdn/vid.jpg
caught=id
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Future/Promise.

Related Patterns

- Contrast Future/Promise with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Future/Promise → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.